

INF-2300: COMPUTER COMMUNICATION

PROJECT 2

Eivind Østlyngen

Eoely

eos033@uit.no

October 14, 2022

1 Introduction

In this report, I will describe and discuss my implementation of the client side of a web application for an online TODO-list solved using HTTP requests.

2 Technical background

2.1 Promises

A promise is an object representing the eventual completion of a asynchronous operation, allowing the user to return asynchronous methods instead of the actual result of the function. The actual result of the function is resolved later to get the result, or it is rejected and returns an error message. Using promises is a powerful way to run code simultaneously while performing large requests.[1]

3 Design & Implementation

To solve this assignment the javascript framework Vue[2] was used to display the website and the javascript library Axios[3] was used to perform the HTTP requests. The client is a simple TODO-list implemented using the basic HTTP requests, PUT, POST, GET and DELETE. These requests are sent using different interactions from the user while interacting with the client. To notify the user of any errors that occurs while performing these requests, both from bad input and from server, an alert on the website with the error message for the server is displayed on any error.

The main HTTP request for the functionality is GET, allowing the user to fetch all saved tasks on the todo-list from the server. The items are received from the server as a list of JSON objects. This request is first performed when launching the site, and subsequently after performing any other request to receive the latest tasks. With the current functionality of the client, as the IDs are not displayed to the user, the endpoint to GET a certain task is not used.

To add another task to the server, the user simply enters the text for the task and clicks a button. After clicking the button, which has a click-listener implemented, the text is stored in a JSON object and sent as the body of a HTTP post request. All other fields for the object is created from the server and do not need to be considered in the client. As mentioned in the previous paragraph, a GET request is performed after each POST, and therefore is the data returned from the response not utilized. The GET request also visually updates the todo-list with the new task. This request should only be limited by the number of tasks that can be stored on the server.

The PUT request is utilized for updating an existing task, both for the completion and the task description. To update an existing task, we simply require the ID for the task. We get this from the task which is interacted with, as all tasks including their ID is stored locally, which allows us to get the complete task with

the updated values for the request body. We can then send the PUT request, replacing the current task with matching ID with the new one. The user should not face any errors if they are the only user of the list, as all tasks shown to the user exists and they have no option choosing the ID to be updated. If they are not the only user, the item they were attempting to change could be deleted.

There are two operations that utilize the DELETE request on the client, deleting a singular task and deleting all existing tasks. Similarly to PUT, we can delete a singular item using the ID of the task. No request body is required as the ID is stored as a parameter in the API URL. For deleting all the task we can simply send the request for all existing IDs.

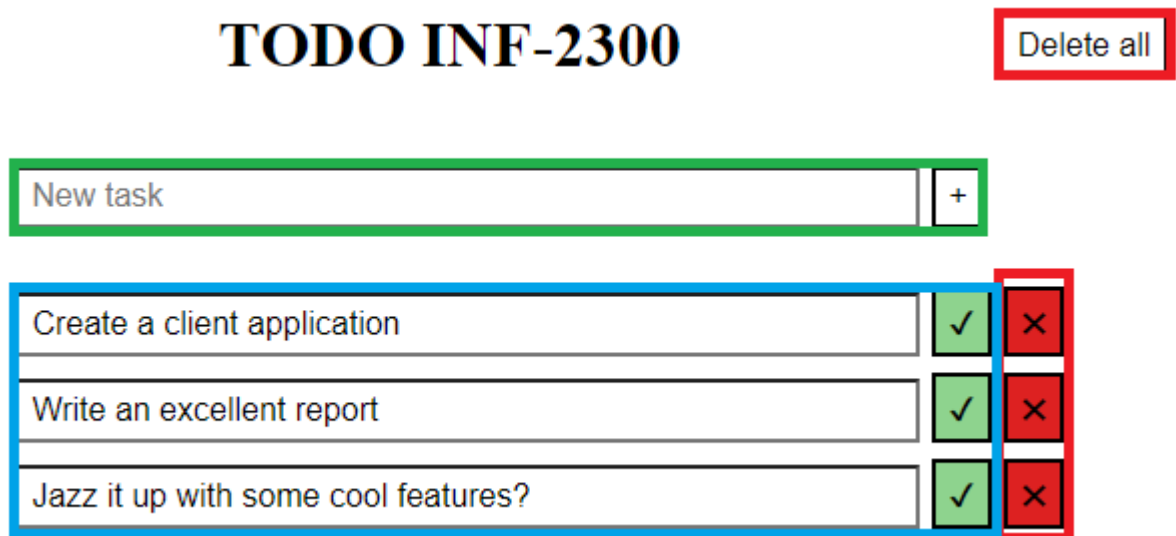


Figure 1: Figure displaying the different HTTP-request from website. DELETE requests are marked with red, PUT blue and POST green.

4 Discussion

4.1 async await

Almost all HTTP-clients to perform requests in javascript are promised based, including axios, and therefore is an important part to understand for implementing a functional client. In this implementation we always resolve the promise straight away as we always fetch all tasks after a request. If we were to GET all tasks before the promise were resolved, we would not get the most updated results as the promises are being resolved simultaneously. This does hinder performance and should only be used when necessary. An example where this could limit the performance greatly is when deleting all the tasks. Deleting a task is not hindered by deleting another task, and therefore we can delete a task without waiting for the previous task to be deleted.

4.2 Multiple users

The implementation currently does not greatly support multiple users using the same list. This is because the changes performed by one user is not visualized by the other user before refreshing the client, while the changes are live in the client. This could lead to users attempting to delete deleted tasks and completing completed tasks. To limit this and get the most up to date data, we get all the tasks after every request. This could also be limited by always fetching the data after a certain period of time. Given the nature of a todo-list this is not an issue as most would have their own todo-list.

4.3 Testing

To test this application I have utilized the included "randomdelay" decorator for the server on all used requests, in case of a slow reply. This does not break the client in any way, as a result of resolving the promise immediately. Even though the client only has input text fields, and no chance of bad input, bad input code was created to test the implementation. This does not break the client or server, and simply displays an error message for the user.

5 Conclusion

In conclusion the implementation meets all the specified requirements and works as expected. The client is user-friendly and has error handling implemented.

6 Sources

References

- [1] Developer.mozilla.org. 2022. Promise - JavaScript — MDN. [online] Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise [Accessed 13 October 2022].
- [2] <https://vuejs.org/> [Accessed 4 October 2022].
- [3] <https://axios-http.com/> [Accessed 4 October 2022].